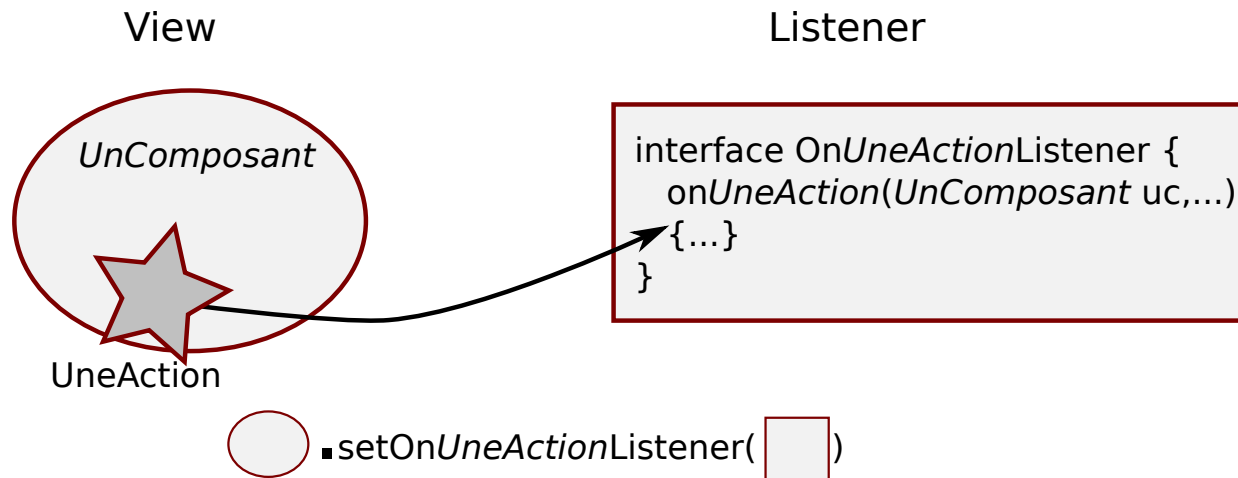


Gestion des événements



- 1 en kotlin, enregistrement d'un écouteur avec `setOnUneActionListener(UnActionListener)` (ou `addUneActionListener`)
 - c'est LA bonne façon de faire
- 2 en XML, utilisation d'un attribut `android:onUneAction`
 - disponible pour certains événements seulement
 - en particulier `android:onClick`
- 3 en kotlin, sous classer pour redéfinir `onUneAction()`
 - à **éviter** dans le cas général
 - pour gestion des événements d'un nouveau composant
 - exemple : `onTouchEvent(MotionEvent)`

Écouteur XML

- utilisation d'un attribut `android:onClick`
- attention à la signature de la méthode : `View` en paramètre, pas de retour
- maintenant déconseillé

```
<Button  
    android:layout_height="wrap_content"  
    android:layout_width="wrap_content"  
    android:text="@string/clic"  
    android:onClick="uneMethode" />
```

```
fun uneMethode(view : View ) {  
    // faire quelque chose  
}
```

Écouteur Kotlin (1)

- enregistrement d'un écouteur avec
`setOnClickListener(OnClickListener)`
- `Ecouteur` = classe d'implantation de la méthode appelée
- Complexité pour rétroagir sur l'interface depuis la classe `Ecouteur` → MVC/MVVM

```
class MonActivite : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_mon)  
  
        val bouton = findViewById<Button>(R.id.button)  
        bouton.setOnClickListener(Ecouteur())  
    }  
}  
  
class Ecouteur : View.OnClickListener {  
    // implantation de la methode demandee  
    override fun onClick(v: View) {  
        // faire quelque chose  
    }  
}
```

Écouteur Kotlin (2)

- enregistrement d'un écouteur avec
`setOnClickListener(OnClickListener)`
- utilisation d'un écouteur anonyme
- à faire en cas d'interface à PLUSIEURS méthodes (rare)

```
val bouton = findViewById<Button>(R.id.button)
bouton.setOnClickListener(object: View.OnClickListener {
    override fun onClick(v: View?) {
        // faire quelque chose
    }
})
```

- La même chose avec une lambda expression
- La méthode à privilégier en cas d'interface fonctionnelle (UNE seule méthode)

```
val bouton = findViewById<Button>(R.id.button)
bouton.setOnClickListener {
    // faire quelque chose
}
```

Écouteur Kotlin : Exemple "complet"

```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        val bouton = findViewById<Button>(R.id.button)  
        bouton.setOnClickListener { action() }  
    }  
  
    private fun action() {  
        Toast.makeText(this, "Ça marche !",  
            Toast.LENGTH_SHORT).show()  
    }  
}
```

Écouteur Kotlin : plusieurs méthodes

```
val editText = findViewById<EditText>(R.id.editText1)

editText.addTextChangedListener(
    object : TextWatcher {
        override fun beforeTextChanged(
            s: CharSequence?, start: Int, count: Int, after: Int) {
            action("1")
        }

        override fun onTextChanged(s: CharSequence?, start: Int,
            before: Int, count: Int) {
            action("2")
        }

        override fun afterTextChanged(s: Editable?) {
            action("3")
        }
    })
}
// ou
editText.addTextChangedListener(
    afterTextChanged = {ed->action(ed.toString())}
)
```

Événements : View

- View super classe des widgets → disponible sur tous les composants
- clic : `onClick()` de l'interface `View.OnClickListener`.
- clic long : `onLongClick()` de `View.OnLongClickListener`.
- changement de focus : `onFocusChange()` de `View.OnFocusChangeListener`.
- clavier : `onKey()` de `View.OnKeyListener` événements claviers
- touché (appui, relâchement, tout mouvement sur l'écran) : `onTouch()` de `View.OnTouchListener`.
- ...

Événements : autres

ListView, GridView :

- **sélection** : `onItemSelected()` de l'interface `AdapterView.OnItemSelectedListener`.

TextView, EditText :

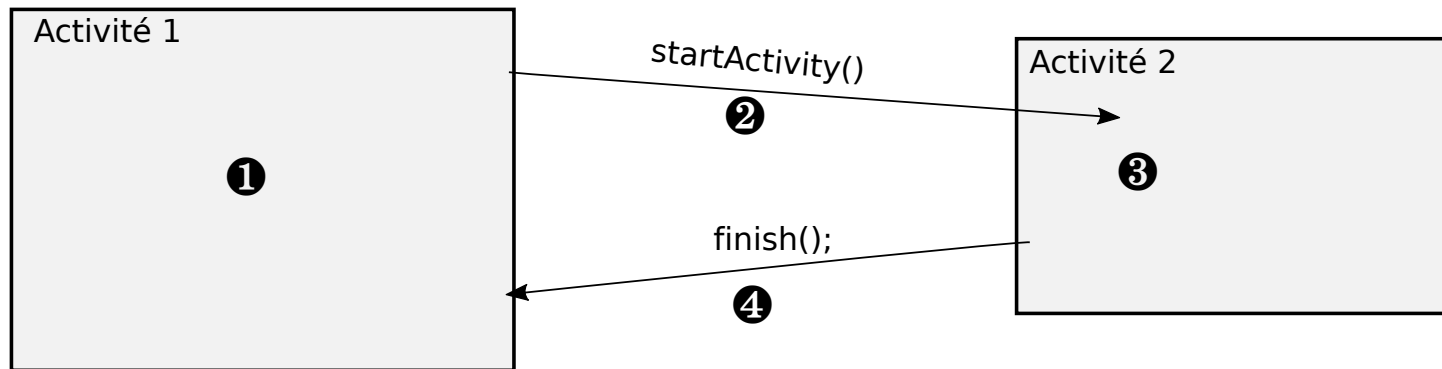
- **modification** : `afterTextChanged()` de `android.text.TextWatcher` (attention ajout de l'écouteur par `addTextChangedListener()`)

...

Communication entre activité : Intent

- nécessité de communiquer entre activités
- compliqué entre applications différentes (pas le même espace mémoire)
- activités (`Activity`), services (`Service`) et récepteurs de diffusion (`BroadcastReceiver`) utilisent des `android.content.Intent`
- Intent :
 - permet de spécifier (de manière souple) une opération à exécuter : prendre une photo, envoyer un sms, ...
 - construite par un composant "demandeur"
 - reçue par une activité qui réalise la demande
 - exprimée de manière :
 - 1 **explicite** en désignant l'activité réceptrice
 - 2 **implicite** en décrivant l'opération souhaitée

Intent : Changement d'activité



- dans Activité 1 :

`startActivity (Intent intent)`

- exemple :

```
val versAutre = Intent(this@MainActivity, SecondActivity::class.java)
startActivity(versAutre)
```

- pour revenir dans Activité 1 depuis Activité 2 :

`finish()` ou *back*

- attention l'activité se déclare dans le manifest

```
<activity android:name=".SecondActivity" > ... </activity>
```

- sinon....

```
android.content.ActivityNotFoundException:
Unable to find explicit activity class {XXX};
have you declared this activity in your AndroidManifest.xml,
or does your intent not match its declared <intent-filter>?
```

Intent

- objet comportant de nombreuses propriétés :
 - 1 action : ce qu'on souhaite du destinataire
 - 2 donnée : données sur lesquelles réaliser l'action
 - 3 type (mime) :
 - 4 catégorie
 - 5 composant (cible) : demande d'une activité particulière → explicite
 - 6 extra : passer des données à l'aide d'une map clé/valeur
 - 7 flags
- un `Intent` est créé et transmis d'une activité à une autre pour communiquer
- toutes les propriétés ne sont pas forcément à renseigner

<http://developer.android.com/reference/android/content/Intent.html>

Intent : structure 1/2

- **action** (*action*) : chaîne représentant l'action à réaliser ACTION_MAIN, ACTION_VIEW, ACTION_DIAL, ACTION_DELETE, ...

```
val newInt = Intent(Intent.ACTION_DIAL)
// ou
val newInt = Intent()
newInt.action = Intent.ACTION_DIAL
```

- **donnée** (*data*) : donnée sur laquelle agir, formatée selon une URI et/ou type MIME

```
val newInt = Intent(
    Intent.ACTION_DIAL,
    Uri.parse("tel:+33240503060")
)
// ou
val newInt = Intent(Intent.ACTION_DIAL)
newInt.data = Uri.parse("tel:+33240503060")
```

Intent : structure 2/2

- catégorie (*category*) : informations supplémentaires sur le composant traitant l'Intent. `CATEGORY_BROWSABLE`, `CATEGORY_LAUNCHER`
- type (*type*) : inféré par les données ou type MIME
- composant (*component*) : en cas d'Intent **explicite** , classe de l'activité à lancer. Utilisation du constructeur

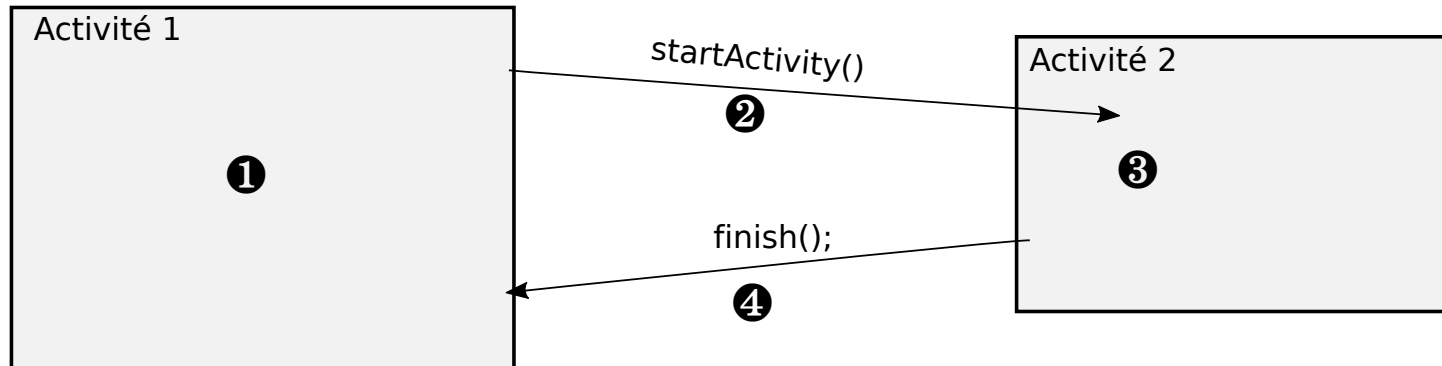
```
uneIntent.type = "image/jpeg"
```

```
Intent (Context packageContext, Class<?> cls)
```

- extras (*extras*) : ajout de données sous forme de clé/valeur

```
unIntent.putExtra(Intent.EXTRA_SUBJECT,  
"Une démo extra !!");
```

Intent explicite



explicite :

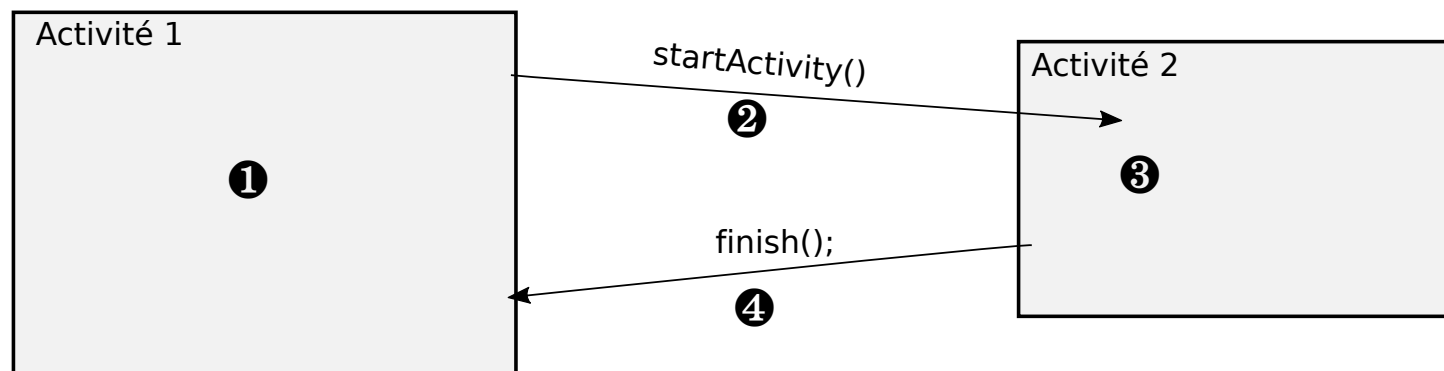
- constructeur permettant de désigner l'activité cible :

```
Intent (Context packageContext, Class<?> cls)
```

- exemple :

```
val versAutre = Intent(this@MainActivity, SecondActivity::class.java)
startActivity(versAutre)
```

Intent implicite



implicite :

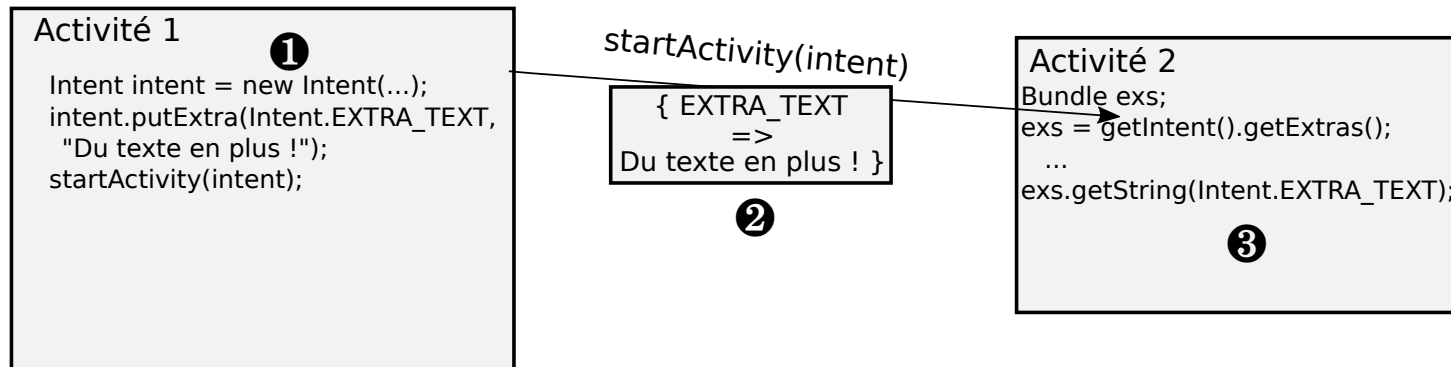
- constructeurs permettant de désigner action et/ou données :

```
Intent (String action)
Intent (String action, Uri uri)
```

- exemple :

```
val c = Calendar.getInstance()
val versAutre = Intent(AlarmClock.ACTION_SET_ALARM)
    .putExtra(AlarmClock.EXTRA_MESSAGE, "Demo")
    .putExtra(AlarmClock.EXTRA_HOUR, c[Calendar.HOUR_OF_DAY])
    .putExtra(AlarmClock.EXTRA_MINUTES, c[Calendar.MINUTE] + 1)
startActivity(versAutre)
```

Intent : avec données



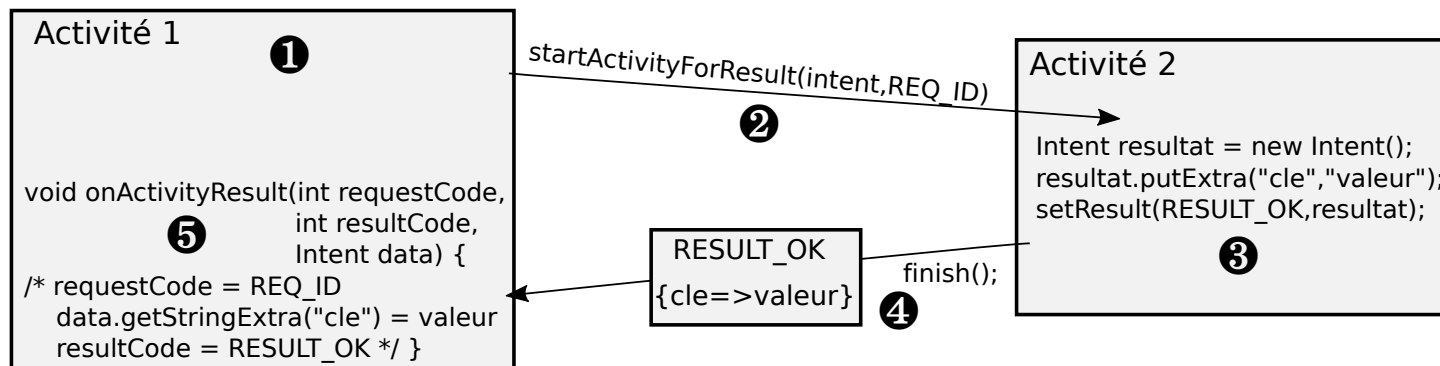
- dans l'Activité 1, on passe une map dans Intent
- utilisation de : `putExtra(clé, valeur)`
clé `String`, valeur type multiples (primitifs, `String`, tableau, `Parcelable`, ...)

```
val intent = Intent(...);  
intent.putExtra(Intent.EXTRA_TEXT, "Du texte en plus !");  
startActivity(intent);
```

- dans l'Activité 2, on accède aux données par :

```
intent.extras?.getString(Intent.EXTRA_TEXT).toString()  
OU  
val valeur2 = intent.getStringExtra(Intent.EXTRA_TEXT)
```


Intent : avec réponse 1/2



- dans l'Activité 1, `startActivityForResult(Intent, int)` plutôt que `startActivity(Intent)`.
- l'entier est un identifiant choisi **ARBITRAIREMENT**

```
val REQUEST_ID = 19;
...
val versAutre = Intent(this, SecondActivity::class.java)
// on peut embarquer des données
// versAutre.putExtra(...);
startActivityForResult(versAutre, REQUEST_ID);
```

Intent : avec réponse 2/2

- dans l'Activité 2, on utilise soit :
 - `setResult(int)` pour un code de retour (dont éventuellement `Activity.RESULT_CANCELED`, `RESULT_OK`)
 - `setResult(int, Intent)` si on veut transmettre un résultat en plus du code de retour.
 - puis `finish()`
- dans l'Activité 1, le retour se fait dans `onActivityResult()`
- `data` est l'Intent réponse de l'Activité 2

```
override fun onActivityResult(requestCode: Int,
                               resultCode: Int,
                               data: Intent?) {
    super.onActivityResult(requestCode, resultCode, data)
    if (requestCode == DEMO) {
        if (resultCode == RESULT_OK) {
            val s = data?.extras?.getInt(SecondActivity.CLE)
            Toast.makeText(this, "reçu : $s", Toast.LENGTH_LONG).show()
        } else
            Toast.makeText(this, "BOUTON BACK ?", Toast.LENGTH_LONG).show()
    }
}
```

Intent : Nouvelle manière 1/2

- l'utilisation de `onActivityResult` est dépréciée !
- à la place : utilisation d'un `Contrat` qui est une sous-classe de `ActivityResultContract<I, O>`. Par ex :
`GetContent` sous classe de `ActivityResultContract<String, Uri?>`

- mise en oeuvre :

- 1 dans l'Activité 1, enregistrement

```
registerForActivityResult<In, Out> (unContrat)
```

- 2 toujours dans cet enregistrement, définition du traitement du résultat.

```
val getContent = registerForActivityResult (GetContent ()) {  
    uri: Uri? -> println(uri)  
}
```

- 3 au moment où le souhaite, lancement de l'Activité 2 : `launch (données)`

```
findViewById<Button> (R.id.boutton) .setOnClickListener {  
    getContent.launch ("image/*")  
}
```

- 4 rien de changé pour Activité 2

Intent : Nouvelle manière 2/2

- Définition d'un contrat personnalisé :
 - 1 sous classer `ActivityResultContract<I, O>`
 - 2 redéfinir
`fun createIntent(context: Context, input: I): Intent`
 - 3 redéfinir
`fun parseResult(resultCode: Int, intent: Intent?): O`
- Il existe un contrat générique `StartActivityForResult` :
(n'importe quel) `Intent` en entrée, `ActivityResult` en sortie (attributs `resultCode` ; `Int` et `data` : `Intent?`)
- Contrats existant : `CaptureVideo`, `PickContact`, `TakePicture`, `RequestPermission`, ...
- `I` et `O` sont à remplacer par les types de votre choix : `String`, `Void`, `Int?`, ...

Intent courants

- quelques combinaisons usuelles
- email : **action** `ACTION_SENDTO`, **extra** `Intent.EXTRA_EMAIL (To)`,
`Intent.EXTRA_SUBJECT`, `Intent.EXTRA_TEXT`
- map : **action** `ACTION_VIEW`, **data** `geo:0,0?q=mon+adresse`
- téléphone : **action** `ACTION_DIAL`, **data** `tel:+numéro`
- photo : **action** `ACTION_IMAGE_CAPTURE`
- contact : **action** `ACTION_PICK`
- ...

<http://developer.android.com/guide/components/intents-common.html>

Intent Filter

- permettent de déclarer les capacités de nos activités
- utilisés dans les résolutions des `Intents` implicites
- déclarés dans le manifest
- critères :

1 action

```
<action android:name="XXX" />
"android.intent.action.AAA" = Intent.ACTION_AAA
"android.intent.action.[PICK|PASTE|ANSWER|...]"
"android.media.action.IMAGE_CAPTURE"
```

2 catégorie

```
<category android:name="XXX" />
android.intent.category.[DEFAULT|BROWSABLE|TAB|ALTERNATIVE
LAUNCHER|HOME|PREFERENCE|TEST]
```

3 données + type : certains aspects de l'URI de la donnée (scheme, host, port, path, etc.) et type MIME

```
<data android:scheme="XXX" android:host="XXX"
    android:port="XXX" android:path="XXX"
    android:pathPattern="XXX"
    android:pathPrefix="strXXXing"
    android:mimeType="XXX" />
```

Intent Filer : exemples

1 cas usuel : activité déclarant une application "principale"

```
<intent-filter>
  <action android:name="android.intent.action.MAIN" />
  <category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
```

2 activité sachant lire et éditer des images png

```
<intent-filter>
  <action android:name="android.intent.action.VIEW" />
  <action android:name="android.intent.action.EDIT" />
  <data android:mimeType="image/png" />
</intent-filter>
```

3 à votre avis ?

```
<intent-filter>
  <action android:name="android.intent.action.VIEW" />
  <data android:type="audio/*" />
  <data android:type="application/ogg" />
  <category android:name="android.intent.category.DEFAULT" />
</intent-filter>
```